31

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

하이브리드와 스탠드얼론

제2부 NestJS 핵심 · 7장 수명주기와 플랫폼

같은 모듈, 다른 입구, 입구 없이



© 한상훈 NestJS 백엔드 마스터

새로 생긴 두 가지 일

→ 둘 다 고객이 부르는 일이 아님

가정

알림 쪽 서비스가 예매 번호로 예매 정보를 물음

가정

매일 새벽, 내일 공연 예매자에게 안내 문자

© 한상훈 NestJS 백엔드 마스터

HTTP로 풀면

→ 내부용 주소 `GET /internal/reservations/:id`: 고객 앱도 부를 수 있는 주소

→ 새벽 배치가 서버에 HTTP 요청: 서버가 떠 있어야 함, 요청 시간 제한에 걸림

→ 배치용 코드를 따로 짜면: 서비스 코드를 복사

© 한상훈 NestJS 백엔드 마스터

홀, 배달, 새벽 반찬



→ 홀 영업: HTTP

→ 배달 주문: 같은 주방, 다른 입구 (하이브리드)

→ 새벽 반찬: 같은 주방, 고객 없이 (스탠드얼론)

© 한상훈 NestJS 백엔드 마스터

실행 형태 셋

→ HTTP 서버: `NestFactory.create`, HTTP 입구 하나

→ 하이브리드 애플리케이션(hybrid application): HTTP 서버에 다른 트랜스포트 입구를 붙인 서버

→ 스탠드얼론 애플리케이션(standalone application): 입구 없이 모듈과 프로바이더만 띄운 앱

→ 셋 다 같은 모듈과 서비스 사용

© 한상훈 NestJS 백엔드 마스터

TCP 입구를 붙인 main.ts

main.ts

```ts
import { type MicroserviceOptions, Transport } from '@nestjs/microservices';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.connectMicroservice<MicroserviceOptions>({
    transport: Transport.TCP,
    options: { host: '127.0.0.1', port: Number(process.env.TCP_PORT ?? 4001) },
  });
  app.enableShutdownHooks();
  await app.startAllMicroservices();
  await app.listen(process.env.PORT ?? 3000);
}
```

→ `npm install @nestjs/microservices`

→ 트랜스포트(transport): 메시지를 실어 나르는 방식 (TCP, Redis, NATS, Kafka 등)

© 한상훈 NestJS 백엔드 마스터

메시지 핸들러

reservations-rpc.controller.ts

```ts
@Controller()
export class ReservationsRpcController {
  constructor(private readonly reservations: ReservationsService) {}

  @MessagePattern('reservations.find')
  find(@Payload() id: number) {
    const reservation = this.reservations.findOne(id);
    if (!reservation) {
      throw new NotFoundException(`예매 ${id}번이 없습니다`);
    }
    return reservation;
  }
}
```

→ ReservationsModule `controllers`에 추가

→ 메시지 패턴: 메시지 이름, HTTP의 경로에 해당

© 한상훈 NestJS 백엔드 마스터

다른 서비스 흉내

scripts/ask-reservation.mjs

```
import { ClientProxyFactory, Transport } from '@nestjs/microservices';
import { firstValueFrom } from 'rxjs';

const client = ClientProxyFactory.create({
  transport: Transport.TCP,
  options: { host: '127.0.0.1', port: Number(process.env.TCP_PORT ?? 4001) },
});
try {
  console.log(await firstValueFrom(client.send('reservations.find', Number(process.argv[2]))));
} catch (err) {
  console.log('에러:', err);
} finally {
  client.close();
}
```

→ scripts/ask-reservation.mjs (시연용 클라이언트)

© 한상훈 NestJS 백엔드 마스터

같은 프로세스, 같은 메모리

터미널

```
$ curl -X POST localhost:4031/reservations ...
{"id":1,"seatId":"A-3-15","approvalId":"pg-50000","amount":50000,"message":"[문자 → 010-1111-2222] A-3-15 예매가 확정되었습니다"}
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 1
{
  id: 1,
  seatId: 'A-3-15',
  buyer: '010-1111-2222',
  approvalId: 'pg-50000',
  userAgent: 'curl/8.16.0'
}
```

→ HTTP로 만든 예매를 TCP로 조회: 같은 ReservationsService 인스턴스

© 한상훈 NestJS 백엔드 마스터

입구마다 다른 주소

```
netstat -ano -p tcp
```

TCP	0.0.0.0:4031	0.0.0.0:0	LISTENING	165612
TCP	127.0.0.1:5131	0.0.0.0:0	LISTENING	165612

→ 같은 PID: 한 프로세스에 입구 둘

→ HTTP 4031: 모든 주소 / TCP 5131: 이 컴퓨터 안에서만

데이터 블록: 시연 포트 HTTP 4031, TCP 5131, 가짜 외부 서버 5031, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

TCP 입구가 먼저 열린 부팅

서버 로그

```
[InstanceLoader] ReservationsModule dependencies initialized +0ms
[NestMicroservice] Nest microservice successfully started +4ms
[SmsOutbox] 발신번호 1588-1234
알림 채널: email, push, sms
[NestApplication] Nest application successfully started +7ms
```

→ TCP 입구: `onModuleInit`(발신번호, 알림 채널)보다 먼저 열림

→ 수명주기 훅은 `app.listen()` 안의 `init`에서 불림

© 한상훈 NestJS 백엔드 마스터

init을 먼저

main.ts

```ts
app.enableShutdownHooks();
await app.init();
await app.startAllMicroservices();
await app.listen(process.env.PORT ?? 3000);
```

서버 로그

```
[SmsOutbox] 발신번호 1588-1234
알림 채널: email, push, sms
[NestApplication] Nest application successfully started +1ms
[NestMicroservice] Nest microservice successfully started +1ms
```

© 한상훈 NestJS 백엔드 마스터

없는 예매를 물으면

터미널

```
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 99
에러: { status: 'error', message: 'Internal server error' }
```

→ 핸들러가 던진 것: `NotFoundException('예매 99번이 없습니다')`

→ 부른 쪽이 받은 것: 문구도 상태 코드도 없음

→ 404: HTTP 응답의 숫자

→ 부른 쪽: 없는 예매인지, 예매 서버가 아픈지 구분 불가

© 한상훈 NestJS 백엔드 마스터

RpcException

reservations-rpc.controller.ts

```ts
import { MessagePattern, Payload, RpcException } from '@nestjs/microservices';

if (!reservation) {
  throw new RpcException({ code: 'RESERVATION_NOT_FOUND', message: `예매 ${id}번이 없습니다` });
}
```

터미널

```
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 99
에러: { code: 'RESERVATION_NOT_FOUND', message: '예매 99번이 없습니다' }
```

→ 부른 쪽이 code로 분기 (없는 예매면 문자를 건너뜀 등)

© 한상훈 NestJS 백엔드 마스터

처리 시간 인터셉터

common/timing.interceptor.ts

```ts
@Injectable()
export class TimingInterceptor implements NestInterceptor {
  private readonly logger = new Logger('Timing');

  intercept(context: ExecutionContext, next: CallHandler): Observable<unknown> {
    const started = performance.now();
    const http = context.switchToHttp();
    const req = http.getRequest<Request>();
    const name = `${context.getClass().name}.${context.getHandler().name}`;
    const done = (result: string) => {
      const ms = (performance.now() - started).toFixed(1);
      this.logger.log(`${req.headers['x-request-id']} ${name} ${result} ${ms}ms`);
    };
    return next.handle().pipe(
      tap({
        next: () => done(String(http.getResponse<Response>().statusCode)),
        error: (err) => done(`실패(${err.constructor.name})`),
      }),
    );
  }
}
```

→ common/timing.interceptor.ts, AppModule { provide: APP_INTERCEPTOR, useClass: TimingInterceptor }

© 한상훈 NestJS 백엔드 마스터

TCP 메시지에는 안 붙음

서버 로그

```
[Timing] 0d0e1920-e516-4a82-940d-8af0483f30e8 AppController.getHello 200 0.4ms
[Timing] 9d7c6779-069a-4ecf-bfd6-556a91b22a92 ReservationsController.reserve 201 19.0ms
```

→ `AppController.getHello`: 띄운 뒤 응답을 확인한 요청

→ TCP로 보낸 `reservations.find` 두 번(1번, 99번): Timing 줄 없음

→ 하이브리드의 TCP 입구: 서버의 전역 설정을 물려받지 않음 (기본)

→ 전역 파이프로 하던 입력 검사도 TCP 쪽에는 없음

© 한상훈 NestJS 백엔드 마스터

inheritAppConfig

main.ts

```
app.connectMicroservice<MicroserviceOptions>({
  transport: Transport.TCP,
  options: { host: '127.0.0.1', port: Number(process.env.TCP_PORT ?? 4001) },
}, { inheritAppConfig: true });
```

터미널

```
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 1
에러: {}
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 99
에러: {}
```

서버 로그

```
ERROR [ProblemDetails] TypeError: Cannot read properties of undefined (reading 'x-request-id')
    at done (.../dist/apps/ticketing/main.js)
    at Object.next (.../dist/apps/ticketing/main.js)
```

→ 물려받는 것: 전역 파이프, 가드, 인터셉터, 예외 필터

→ TCP 메시지에서 `switchToHttp().getRequest()`: HTTP 요청 대신 메시지 데이터(숫자 1)

→ 핸들러는 1번을 찾음, 끝난 뒤 시간을 찍는 줄에서 TypeError

→ 그 에러를 같이 물려받은 ProblemDetailsFilter가 받음, HTTP 응답을 만들다 깨짐

© 한상훈 NestJS 백엔드 마스터

getType으로 나눈 인터셉터

common/timing.interceptor.ts

```
const name = `${context.getClass().name}.${context.getHandler().name}`;
const isHttp = context.getType() === 'http';
const http = context.switchToHttp();
const tag = isHttp ? http.getRequest<Request>().headers['x-request-id'] : context.getType();
// ...
next: () => done(isHttp ? String(http.getResponse<Response>().statusCode) : 'ok'),
```

터미널

```
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 1
{
  id: 1,
  seatId: 'A-3-15',
  buyer: '010-1111-2222',
  approvalId: 'pg-50000',
  userAgent: 'curl/8.16.0'
}
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 99
에러: {}
```

서버 로그

```
[Timing] rpc ReservationsRpcController.find ok 0.3ms
[Timing] rpc ReservationsRpcController.find 실패(RpcException) 0.4ms
```

→ 99번 서버 로그: ProblemDetails의 ERROR, 받은 예외는 RpcException(예매 99번 문구)

→ `getType()`: `'http'`, `'rpc'`, `'ws'`

→ 99번: 모든 예외를 받는 ProblemDetailsFilter(`@Catch()`)가 RpcException까지 받음

© 한상훈 NestJS 백엔드 마스터

HTTP 필터는 HTTP 요청만

problem-details.filter.ts

```ts
@Catch()
export class ProblemDetailsFilter implements ExceptionFilter {
  private readonly rpcFilter = new BaseRpcExceptionFilter();

  catch(exception: unknown, host: ArgumentsHost) {
    if (host.getType() !== 'http') return this.rpcFilter.catch(exception, host);
    // ... Problem Details 응답 (그대로)
```

터미널과 서버 로그

```
$ TCP_PORT=5131 node scripts/ask-reservation.mjs 99
에러: { code: 'RESERVATION_NOT_FOUND', message: '예매 99번이 없습니다' }
[Timing] rpc ReservationsRpcController.find 실패(RpcException) 0.3ms
```

→ BaseRpcExceptionFilter(`@nestjs/microservices`): RpcException은 그 내용 그대로, 그 밖은 Internal server error

→ DomainErrorFilter에도 같은 줄

→ 직원 키 가드: Roles 표시가 없으면 요청 객체를 열기 전에 통과, 손댈 것 없음

→ 프로젝트: inheritAppConfig 켬, 인터셉터와 필터 둘을 나눈 코드

© 한상훈 NestJS 백엔드 마스터

입구를 하나 더 열면

"입구를 하나 더 열면, 그 입구의 예외와 전역 설정을 다시 정해야 됩니다."

© 한상훈 NestJS 백엔드 마스터

안내 문자 배치 reminders.ts

reminders.ts

```ts
import { NestFactory } from '@nestjs/core';
import { readFile } from 'node:fs/promises';
import { NotificationsModule } from './notifications/notifications.module.js';
import { NotificationsService } from './notifications/notifications.service.js';

process.loadEnvFile();

const app = await NestFactory.createApplicationContext(NotificationsModule);
const notifications = app.get(NotificationsService);
const list = JSON.parse(await readFile(process.argv[2], 'utf8'));
for (const r of list) notifications.sendReminder(r.buyer, r.seatId, r.startsAt);
console.log(`안내 문자 ${list.length}건 접수`);
await app.close();
```

→ `NotificationsService.sendReminder`: 문자 채널로 '내일 19:00 공연, 좌석 A-3-15'

→ 예매 목록: 파일로 받음 (DB 전이라서)

© 한상훈 NestJS 백엔드 마스터

빌드와 실행

nest-cli.json

```
"reminders": {
  "type": "application",
  "root": "apps/reminders",
  "entryFile": "reminders",
  "sourceRoot": "apps/ticketing/src",
  "compilerOptions": {
    "tsConfigPath": "apps/ticketing/tsconfig.app.json",
    "deleteOutDir": false
  }
}
```

터미널

```
npx nest build reminders
node dist/apps/reminders/reminders.js reminders.json
```

→ nest-cli.json projects에 진입점 하나 추가,
소스는 예매 서버와 같은 폴더

© 한상훈 NestJS 백엔드 마스터

안내 문자 세 건

터미널

```
15:07:23.739 $ node dist/apps/reminders/reminders.js reminders.json
[InstanceLoader] DiscoveryModule dependencies initialized +15ms
[InstanceLoader] NotificationsModule dependencies initialized +0ms
[SmsOutbox] 발신번호 1588-1234
알림 채널: email, push, sms
안내 문자 3건 접수
[SmsOutbox] 문자 3건 보냄: 200
[SmsOutbox] onApplicationShutdown(undefined): 남은 문자 보냄
15:07:24.950 종료
```

발송사 로그

```
15:07:24.881 [문자] 발송 3건
    1588-1234 → 010-1111-2222 "내일 19:00 공연, 좌석 A-3-15"
    1588-1234 → 010-3333-4444 "내일 19:00 공연, 좌석 A-3-16"
    1588-1234 → 010-5555-6666 "내일 19:00 공연, 좌석 B-1-7"
```

→ HTTP 포트 없음, 수명주기 훅은 그대로

→ `onApplicationShutdown`의 signal: close로 끝내면 `undefined`

© 한상훈 NestJS 백엔드 마스터

AppModule로 띄운 배치

reminders.ts

```
const app = await NestFactory.createApplicationContext(AppModule);
console.log('예매 1번:', app.get(ReservationsService).findOne(1));
const notifications = app.get(NotificationsService);
```

→ 처음 짠 코드: 서버 전체를 띄우고 필요한 서비스를 꺼냄

→ AppModule 안: 결제 모듈(부팅 때 결제사 토큰, 3초 제한) 포함

© 한상훈 NestJS 백엔드 마스터

결제사 점검 시간

터미널

```
15:08:00.301 $ node dist/apps/reminders/reminders.js reminders.json
결제 설정: {"mode":"http","baseUrl":"http://localhost:5031","timeoutMs":3000}
[Nest] 오후 3:08:01  LOG [InstanceLoader] SeatsModule dependencies initialized +9ms
[Nest] 오후 3:08:04  ERROR [ExceptionHandler] DOMException [TimeoutError]: The operation was aborted due to timeout
15:08:04.141 종료
```

→ 배치가 도는 새벽에 결제사 정기 점검 (가정)

→ 가짜 결제사: 토큰 요청에 답하지 않음 (점검 흉내)

→ 문자 발송사 기록: 없음

© 한상훈 NestJS 백엔드 마스터

서버가 만든 예매 1번

터미널

```
$ curl -X POST localhost:4031/reservations ...
{"id":1,"seatId":"A-3-15",...}
15:08:06.188 $ node dist/apps/reminders/reminders.js reminders.json
예매 1번: undefined
안내 문자 3건 접수
```

→ 스탠드얼론 앱: 서버와 다른 프로세스, 다른 메모리

→ 하이브리드의 TCP 입구: 같은 프로세스, 같은 메모리

© 한상훈 NestJS 백엔드 마스터

close 없는 배치

터미널

```
15:07:30.541 $ node dist/apps/reminders/reminders.js reminders.json
[SmsOutbox] 발신번호 1588-1234
안내 문자 3건 접수
[Nest] 오후 3:07:41  LOG [SmsOutbox] 문자 3건 보냄: 200
15:07:45.675 종료 (timeout 15초가 끊음)
```

→ 마지막 줄 `await app.close()`를 지운 코드

→ 문자: 10초 타이머가 보냄 / 프로세스: 타이머가 살아 있어서 안 끝남

© 한상훈 NestJS 백엔드 마스터

close를 부른 배치

	close 없음	close 있음
끝나는 시각	15초 뒤 외부에서 끊음	1.2초
문자 발송	10초 타이머	`onApplicationShutdown`
타이머	살아 있음	`clearInterval`

→ 매일 도는 배치: 안 끝나면 다음 날 실행과 겹침, 스케줄러에는 실패로 기록 (가정)

→ 중간 에러에도 close: `try finally` (예: 깨진 예매 목록 파일의 `JSON.parse` 에러)

© 한상훈 NestJS 백엔드 마스터

세 가지 실행 형태

	띄우는 함수	입구
HTTP 서버	`NestFactory.create + listen`	HTTP
하이브리드	`create + connectMicroservice + init + startAllMicroservices + listen`	HTTP와 TCP 등
스탠드얼론	`NestFactory.createApplicationContext + close`	없음

→ 모듈, 서비스, 수명주기 훅: 셋 모두 같음

→ 고르는 기준, 누가 시작하나: 고객·결제사 웹훅은 HTTP, 내부 서비스는 하이브리드, 정해진 시각에 한 번은 스탠드얼론

© 한상훈 NestJS 백엔드 마스터

HTTP가 필요 없는 일

"HTTP가 필요 없는 일은 필요한 모듈만 불러서 스탠드얼론 앱으로 돌리고, 끝나면 close를 부릅니다."